

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

**UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM - 631 561**

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



Name : T.SAI TEJA

Reg.No : 11249A374

Class : IIB.E(CSE)

Subject Code : BCSF2441A12

Subject Name : DATA SCIENCE TOOLKIT LAB

**SRI CHANDRASEKHARENDRA SARASWATHI
VISWA MAHAVIDYALAYA**

**UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM - 631 561**

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by.

Mr/Ms. T.SAITEJA with Reg.No. 11249A374 done by of II-B.E.(CSE) in (BCSF2441A12 DATA SCIENCE TOOLKIT LAB).for Lab during the academic year 2025 -2026.

Station : ENATHUR

Date :

**Staf-in-charge
Head of the Department**

Submitted for the Practical examination held on _____

**Examine-1
Examine -2**

INDEX

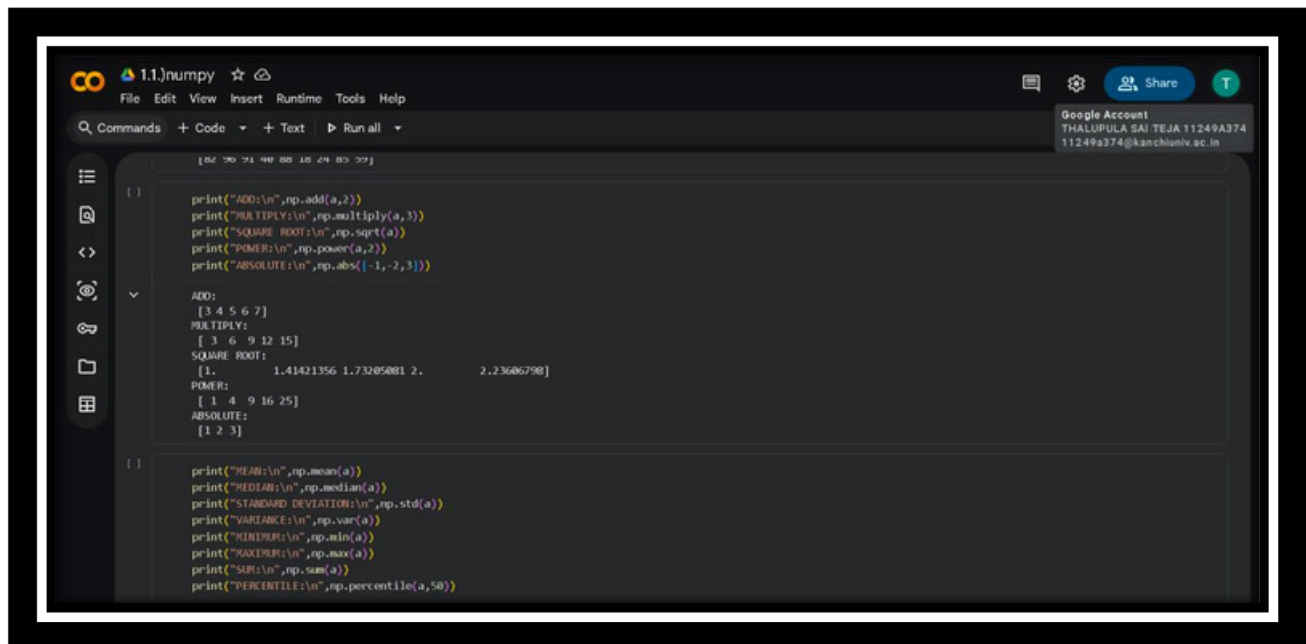
Sl.NO	Program Name	Date 02/02/2026	Page .no	Signature
01 (1.1)	Introduction to Python Libraries for Data Science			
1.2	Pandas	02/02/2026		
<u>1.3</u>	<u>Matplotlib</u>	02/02/2026		
2	Perform DataExploration and Pre-processing in Python Implement Regularized	<u>09/02/2026</u>		
3	Linear Regression	<u>14/02/2026</u>		
4	Implement Naive Bayes Classifier	23/02/2026		
5	Implement Regularized Logistic Regression	<u>02/03/2026</u>		
6	Build Modules Using Different Ensembling Techniques	09/03/2026		
7	Build Models Using Decision Trees	16/03/2026		
8	Build the Models Using Different Kernel Functions	23/03/2026		
9	Implement K-Nearest Neighbours (KNN) Classification	23/03/2026		
10	K-Mean Clustering with PCA and Elbow Method	23/03/2026		

INTRODUCTION TO PYTHON LIBRARIES FOR DATASCIENCE

EXP.NO:-1.1

DATE:-02/02/2026

Aim: To understand the basic Python libraries used in data science, such as NumPy, Pandas, Matplotlib, and Scikit-learn.



A screenshot of a Jupyter Notebook interface. The top bar shows the Jupyter logo, the file name '1.1.numpy', and a star icon. Below the bar is a menu with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A search bar with 'Commands' and a dropdown menu with '+ Code', '+ Text', and 'Run all' is visible. The main area contains two code cells. The first cell has the following code:

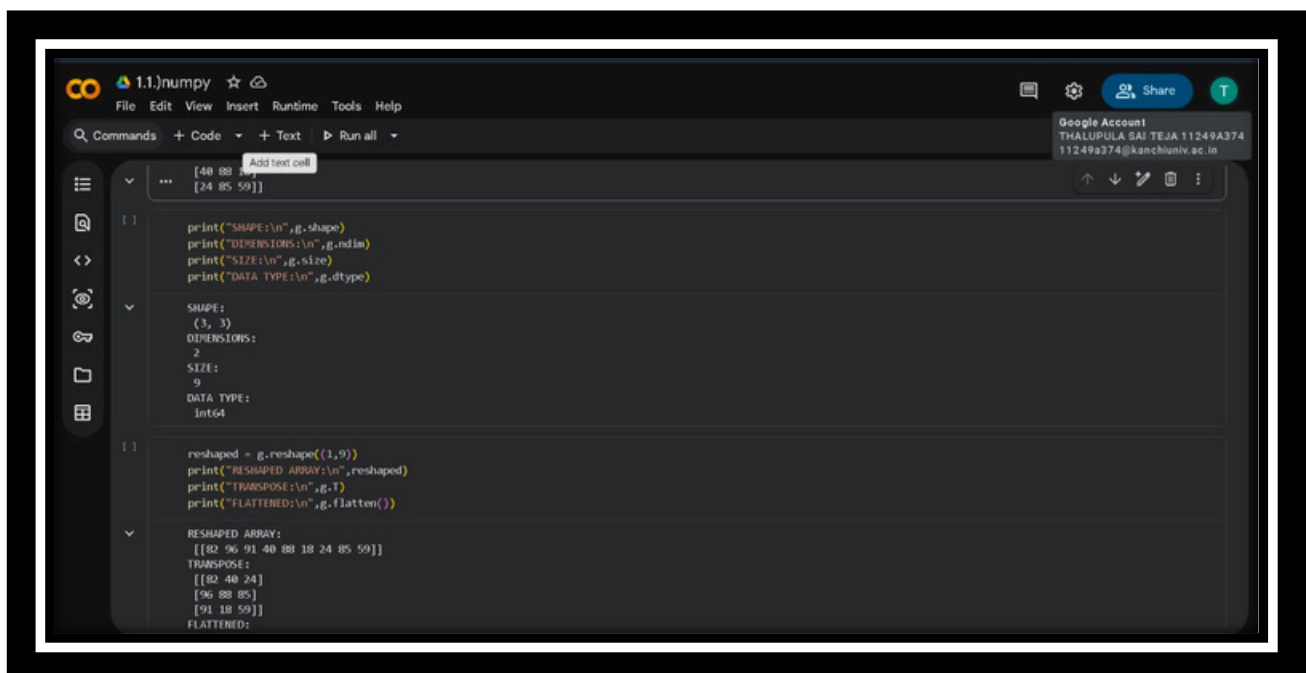
```
[0. 3. 4. 5. 6. 7.]  
  
[ ]  
print("ADD:\n",np.add(a,2))  
print("MULTIPLY:\n",np.multiply(a,3))  
print("SQUARE ROOT:\n",np.sqrt(a))  
print("POWER:\n",np.power(a,2))  
print("ABSOLUTE:\n",np.abs([-1,-2,3]))
```

 The output of this cell is:

```
ADD:  
[3 4 5 6 7]  
MULTIPLY:  
[ 3  6  9 12 15]  
SQUARE ROOT:  
[1.          1.41421356 1.73205081 2.          2.23606798]  
POWER:  
[ 1  4  9 16 25]  
ABSOLUTE:  
[1 2 3]
```

 The second cell has the following code:

```
[ ]  
print("MEAN:\n",np.mean(a))  
print("MEDIAN:\n",np.median(a))  
print("STANDARD DEVIATION:\n",np.std(a))  
print("VARIANCE:\n",np.var(a))  
print("MINIMUM:\n",np.min(a))  
print("MAXIMUM:\n",np.max(a))  
print("SUM:\n",np.sum(a))  
print("PERCENTILE:\n",np.percentile(a,50))
```



A screenshot of a Jupyter Notebook interface. The top bar shows the Jupyter logo, the file name '1.1.numpy', and a star icon. Below the bar is a menu with 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. A search bar with 'Commands' and a dropdown menu with '+ Code', '+ Text', and 'Run all' is visible. The main area contains two code cells. The first cell has the following code:

```
[ ]  
print("SHAPE:\n",g.shape)  
print("DIMENSIONS:\n",g.ndim)  
print("SIZE:\n",g.size)  
print("DATA TYPE:\n",g.dtype)
```

 The output of this cell is:

```
SHAPE:  
(3, 3)  
DIMENSIONS:  
2  
SIZE:  
9  
DATA TYPE:  
int64
```

 The second cell has the following code:

```
[ ]  
reshaped = g.reshape((1,9))  
print("RESHAPED ARRAY:\n",reshaped)  
print("TRANPOSE:\n",g.T)  
print("FLATTENED:\n",g.flatten())
```

 The output of this cell is:

```
RESHAPED ARRAY:  
[[92 96 91 40 88 18 24 85 59]]  
TRANPOSE:  
[[92 40 24]  
 [96 88 85]  
 [91 18 59]]  
FLATTENED:
```

```
1.1.numpy
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Google Account
THALUPULA SAI TEJA 11249A374
11249a374@kanchiuniv.ac.in

[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15]

1.1
print("ADD:\n",np.add(a,2))
print("MULTIPLY:\n",np.multiply(a,3))
print("SQUARE ROOT:\n",np.sqrt(a))
print("POWER:\n",np.power(a,2))
print("ABSOLUTE:\n",np.abs([-1,-2,3]))

ADD:
[ 3  4  5  6  7]
MULTIPLY:
[ 3  6  9 12 15]
SQUARE ROOT:
[1.         1.41421356 1.73205081 2.         2.23606798]
POWER:
[ 1  4  9 16 25]
ABSOLUTE:
[1 2 3]

1.1
print("MEAN:\n",np.mean(a))
print("MEDIAN:\n",np.median(a))
print("STANDARD DEVIATION:\n",np.std(a))
print("VARIANCE:\n",np.var(a))
print("MINIMUM:\n",np.min(a))
print("MAXIMUM:\n",np.max(a))
print("SUM:\n",np.sum(a))
print("PERCENTILE:\n",np.percentile(a,50))
```

```
1.1.numpy
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Google Account
THALUPULA SAI TEJA 11249A374
11249a374@kanchiuniv.ac.in

5.0
MEDIAN:
3.0
STANDARD DEVIATION:
1.4142135623730951
VARIANCE:
2.0
MINIMUM:
1
MAXIMUM:
5
SUM:
15
PERCENTILE:
3.0

1.1
print("COLUMN-WISE SUM:\n",np.sum(g,axis=0))
print("ROW-WISE MEAN:\n",np.mean(g,axis=1))

COLUMN-WISE SUM:
[146 269 168]
ROW-WISE MEAN:
[09.66666667 48.66666667 56. ]

1.1
print("FIRST ELEMENT:\n",a[0])
print("SLICE:\n",a[1:4])
print("2D SLICE:\n",g[0:2,1:3])

FIRST ELEMENT:
5
```

A screenshot of a Jupyter Notebook interface with a dark theme. The top bar shows '1.1.numpy' and standard menu items. The left sidebar contains icons for file explorer, search, and other tools. The main area displays three code cells. The first cell shows a 2D array 'g' and its slices. The second cell performs logical operations on 'g' using 'np.where', 'np.any', and 'np.all'. The third cell sorts an array 'a' and finds its maximum, minimum, and unique elements. The output of each cell is displayed below the code.

```
Slice:  
[2 3 4]  
2D Slice:  
[[96 91]  
 [88 18]]  
  
[ ]  
print("WHERE > 50:\n",np.where(g>50))  
print("ANY > 50:\n",np.any(g>90))  
print("ALL > 0:\n",np.all(g>0))  
  
WHERE > 50:  
(array([0, 0, 0, 1, 2, 2]), array([0, 1, 2, 1, 1, 2]))  
ANY > 50:  
True  
ALL > 0:  
True  
  
[ ]  
print("SORTED a:\n",np.sort(a))  
print("INDEX OF MAX:\n",np.argmax(a))  
print("INDEX OF MIN:\n",np.argmin(a))  
print("UNIQUE ELEMENTS:\n",np.unique([1,2,2,4,4]))  
  
SORTED a:  
[1 2 3 4 5]  
INDEX OF MAX:  
4  
INDEX OF MIN:  
0  
UNIQUE ELEMENTS:  
[1 2 3 4 5]
```

A screenshot of a Jupyter Notebook interface with a dark theme. The top bar shows '1.1.numpy' and standard menu items. The left sidebar contains icons for file explorer, search, and other tools. The main area displays a single code cell that creates two 2D arrays 'x' and 'y' and calculates their dot product using 'np.dot'. The output shows the resulting 2x2 matrix.

```
[1 2 4]  
  
[ ]  
x = np.array([[1,2],[3,4]])  
y = np.array([[5,6],[7,8]])  
print("DOT PRODUCT:\n",np.dot(x,y))  
  
DOT PRODUCT  
: [[19 22]  
 [43 50]]
```

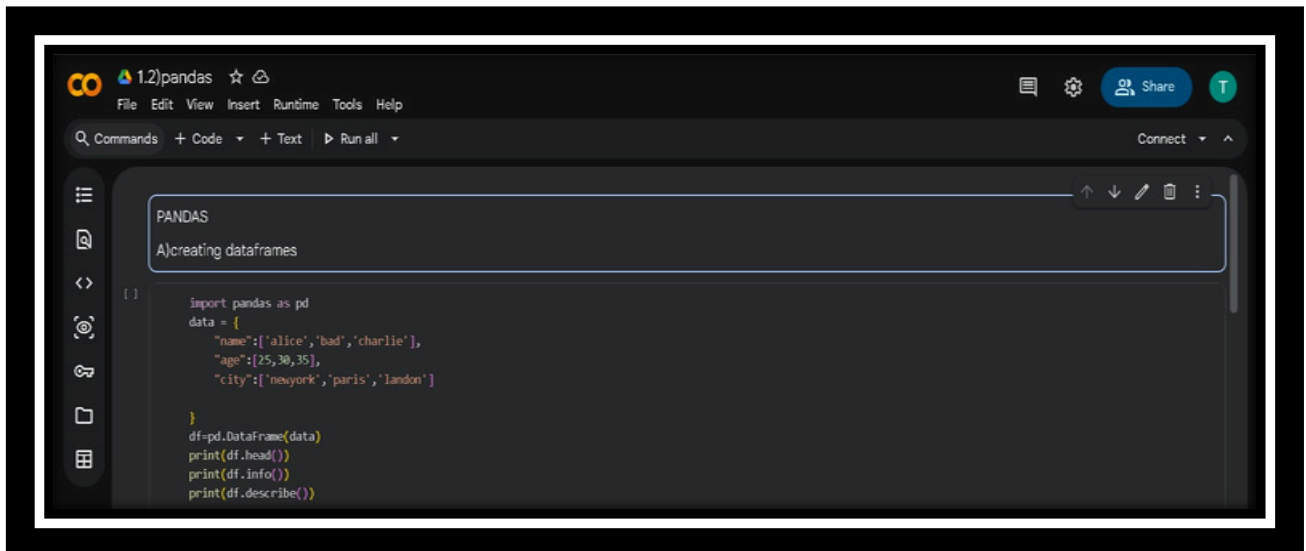
Result: Understood the role of Python libraries in handling, analyzing, and visualizing data. Python libraries such as NumPy, Pandas, Matplotlib, and Scikit-learn are commonly used for these tasks.

EXP.NO:-1.2

DATE:-02/02/2026

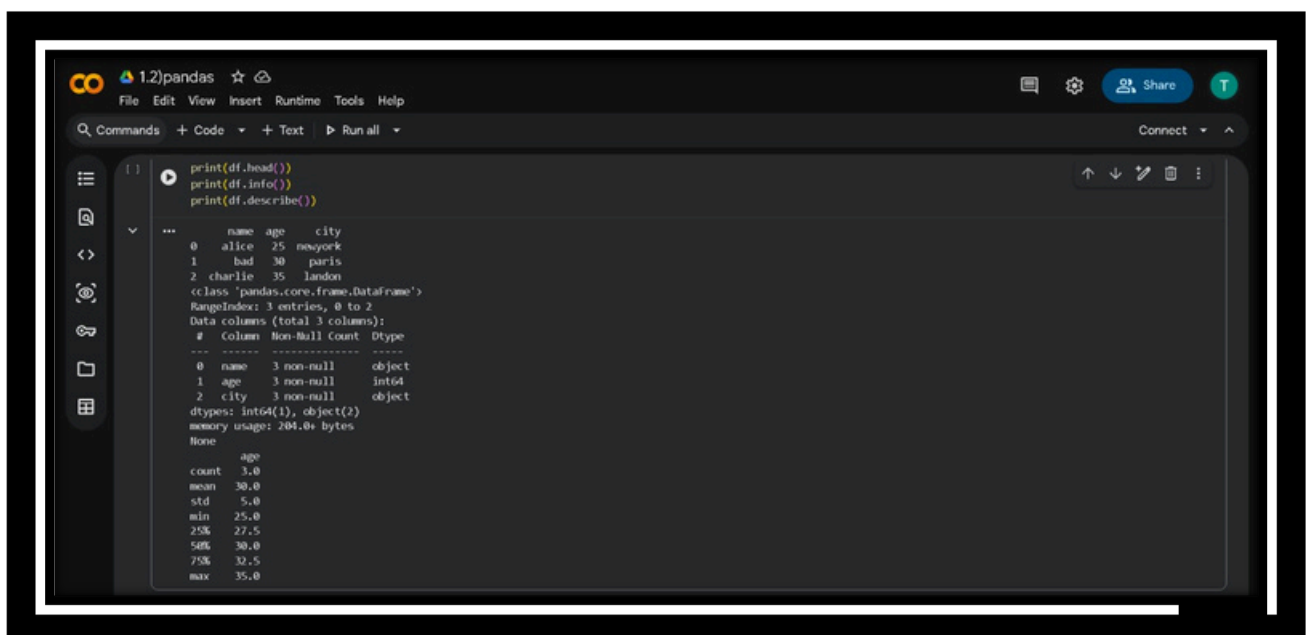
Pandas

Aim: To learn how to use Pandas for data loading, manipulation, and analysis.



The screenshot shows a Jupyter Notebook interface with a dark theme. The top bar includes the Jupyter logo, the text "1.2)pandas", and icons for star, share, and a terminal. Below the top bar is a menu bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". A search bar with "Commands" and a "Run all" button are also present. The left sidebar contains icons for file explorer, search, and other notebook functions. The main area has a title "PANDAS" and a subtitle "A)creating dataframes". The code cell contains the following Python code:

```
[ ]
import pandas as pd
data = {
    "name":['alice','bad','charlie'],
    "age":[25,30,35],
    "city":['newyork','paris','london']
}
df=pd.DataFrame(data)
print(df.head())
print(df.info())
print(df.describe())
```



The screenshot shows the same Jupyter Notebook interface, but now the code cell is executed, and the output is displayed. The output includes the head of the DataFrame, the information of the DataFrame, and the descriptive statistics of the DataFrame.

```
[ ]
print(df.head())
print(df.info())
print(df.describe())
```

```
...
```

	name	age	city
0	alice	25	newyork
1	bad	30	paris
2	charlie	35	london

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0  name    3 non-null    object
1  age     3 non-null    int64
2  city    3 non-null    object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
```

	age
count	3.0
mean	30.0
std	5.0
min	25.0
25%	27.5
50%	30.0
75%	32.5
max	35.0

The screenshot shows a Jupyter Notebook interface with the title '1.2)pandas'. The menu bar includes File, Edit, View, Insert, Runtime, Tools, and Help. The toolbar has options for Commands, Code, Text, and Run all. The notebook is titled 'B)SELECTING AND FILTERING'. The code cell contains the following Python code:

```
ages = df["age"]
above_25 = df[df['age']>25]
row_0=df.iloc[0]
specific_val=df.loc[0,'city']

print(ages)
print(above_25)
print(row_0)
print(specific_val)
```

The output of the code is displayed below the code cell:

```
0    25
1    30
2    35
Name: age, dtype: int64
name age city
1    bad  30  paris
2  charlie  35  london
name    alice
age      25
city  newyork
Name: 0, dtype: object
newyork
```

The screenshot shows a Jupyter Notebook interface with the title '1.2)pandas'. The menu bar includes File, Edit, View, Insert, Runtime, Tools, and Help. The toolbar has options for Commands, Code, Text, and Run all. The notebook is titled 'C)Data Cleaning(Handling Missing Values)'. The code cell contains the following Python code:

```
print(df.isnull().sum())
df_clean=df.dropna()
df_filled=df.fillna(0)
df['age']=df['age'].fillna(df['age'].mean())
print(df['age'])
```

The output of the code is displayed below the code cell:

```
name    0
age     0
city    0
dtype: int64
0    25
1    30
2    35
Name: age, dtype: int64
```

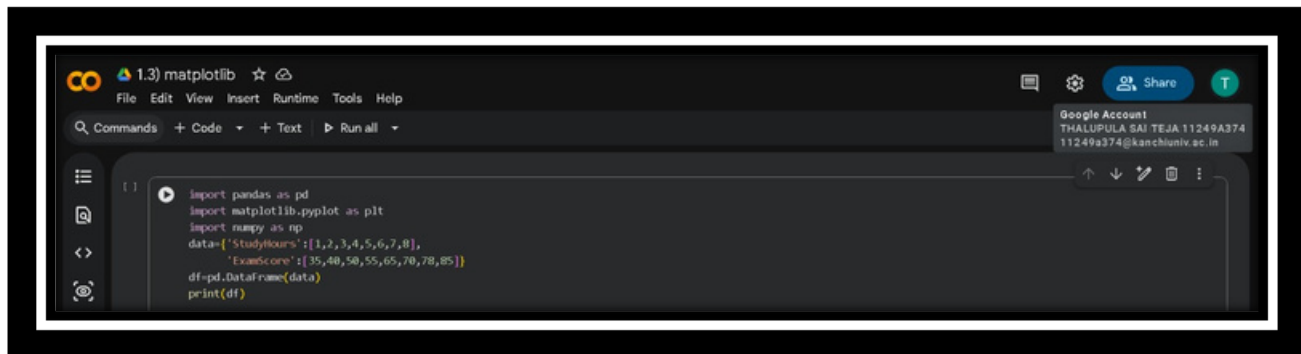
Result: Successfully performed data manipulation tasks such as creating, modifying, cleaning, filtering, and summarizing datasets. Pandas is widely used for preparing data for analysis and machine learning.

EXP.NO:-1.3

DATE:-02/02/2026

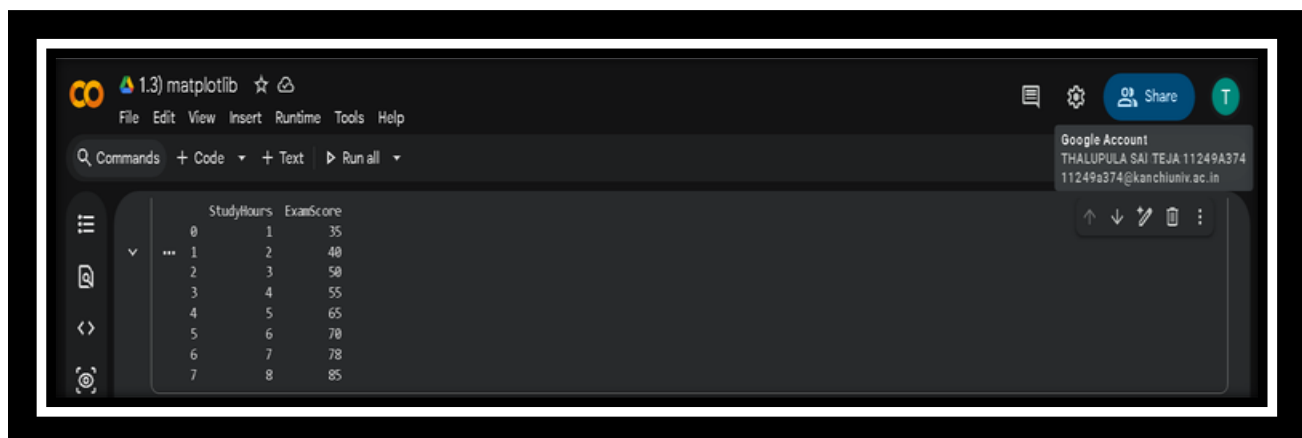
Matplotlib

Aim: To learn how to create graphical visualizations using Matplotlib.



```
1.3) matplotlib ☆ ☁  
File Edit View Insert Runtime Tools Help  
Q Commands + Code + Text ▶ Run all  
Google Account  
THALUPULA SAI TEJA 11249A374  
11249a374@kanchiuniv.ac.in  
import pandas as pd  
import matplotlib.pyplot as plt  
import numpy as np  
data={'StudyHours':[1,2,3,4,5,6,7,8],  
      'ExamScore':[35,40,50,55,65,70,78,85]}  
df=pd.DataFrame(data)  
print(df)
```

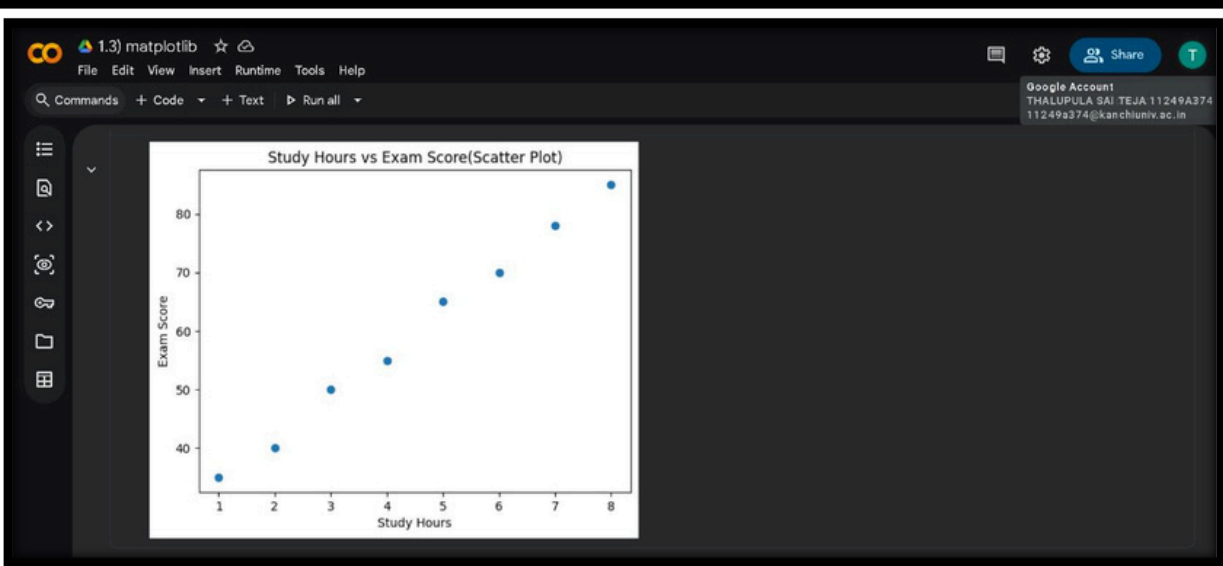
output:-



	StudyHours	ExamScore
0	1	35
1	2	40
2	3	50
3	4	55
4	5	65
5	6	70
6	7	78
7	8	85

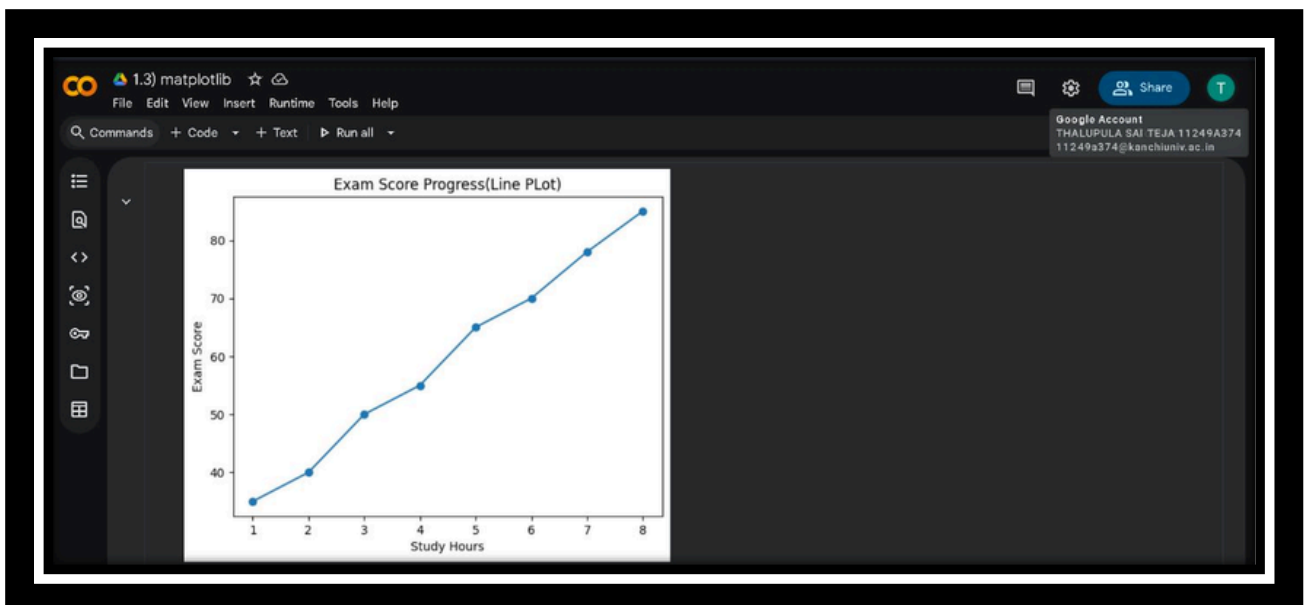
```
1.3) matplotlib ☆
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
1 scatter plot
[ ]
plt.scatter(df['StudyHours'],df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score(Scatter Plot)")
plt.show()
```

output:-



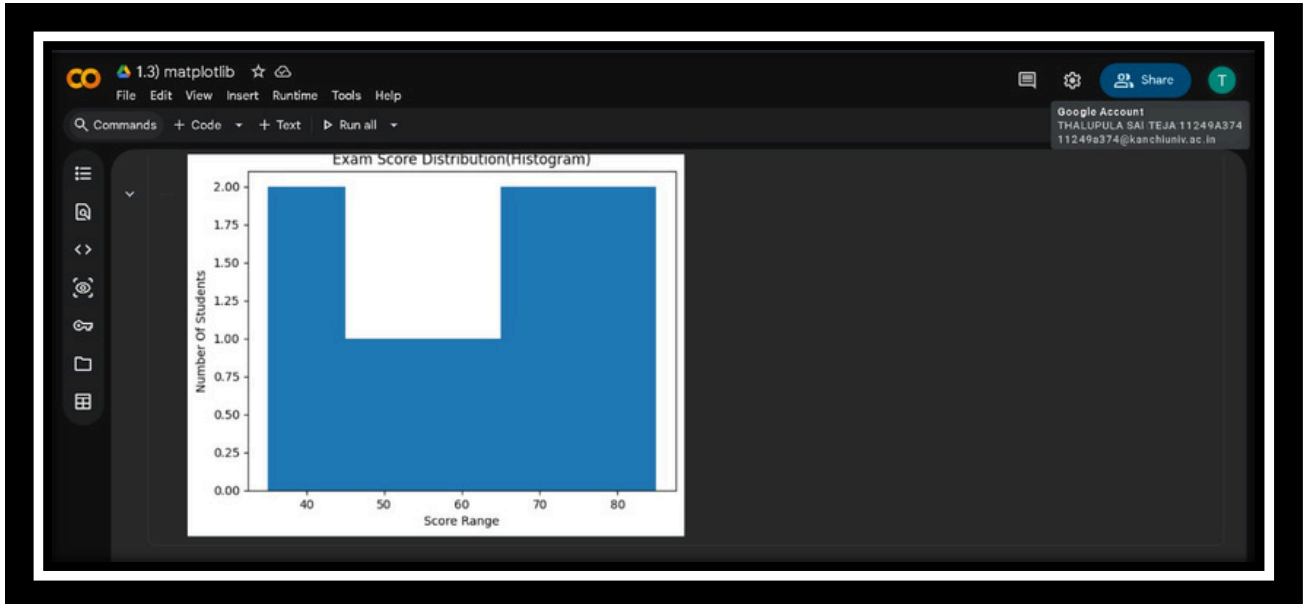
```
1.3) matplotlib ☆ ☁
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all
2 line plot
[ ]
plt.plot(df['StudyHours'],df['ExamScore'],marker='o')
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Exam Score Progress(Line Plot)")
plt.show()
```

output:-



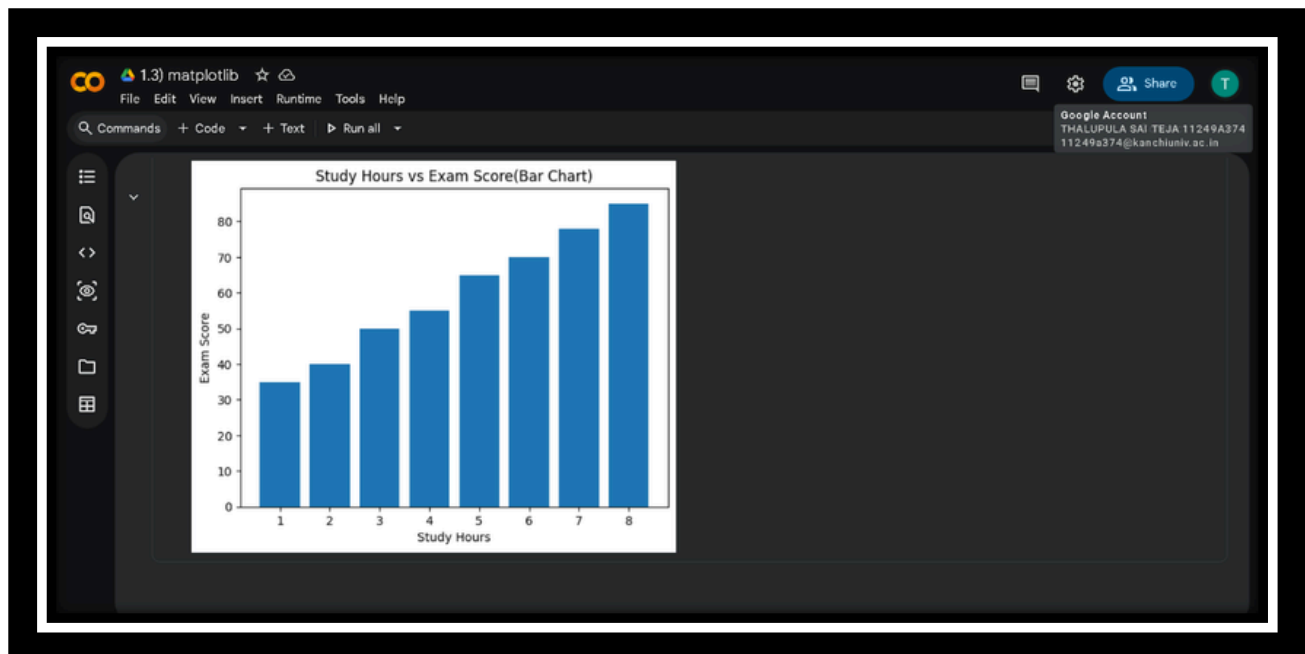
```
1.3) matplotlib ☆
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
3 histogram
1:1
plt.hist(df['ExamScore'],bins=5)
plt.xlabel("Score Range")
plt.ylabel("Number Of Students")
plt.title("Exam Score Distribution(Histogram)")
plt.show()
```

output:-



```
1.3) matplotlib
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text Run all
4 bar chart
plt.bar(df['StudyHours'],df['ExamScore'])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Hours vs Exam Score(Bar Chart)")
plt.show()
```

output:-



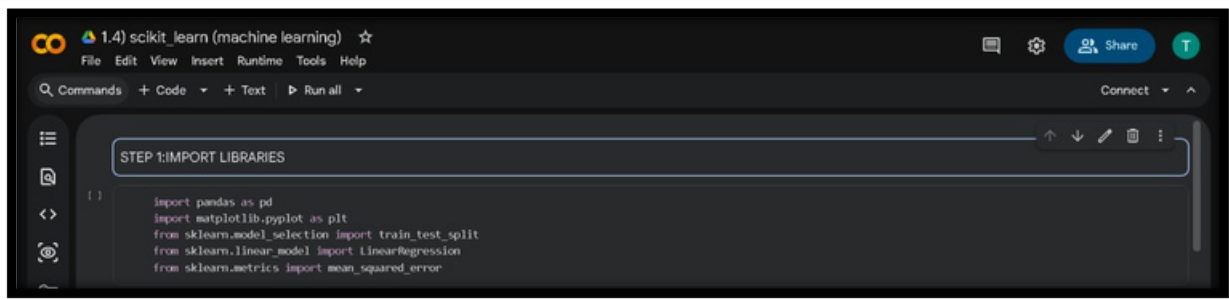
Result: Successfully created plots such as line charts, bar charts, scatter plots, and other basic graphs for data presentation. Matplotlib is commonly used for visualizing x and y points and customizing plots

EXP.NO:-1.4

DATE:-09/02/2026

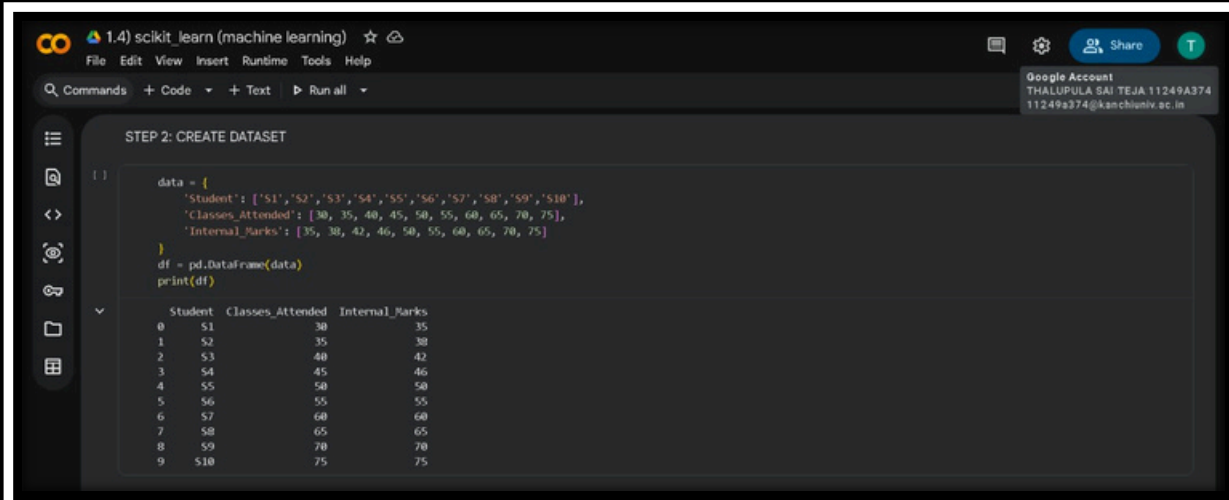
Scikit-learn

Aim: To understand the use of Scikit-learn for building machine learning models.



The screenshot shows a Jupyter Notebook interface with the title '1.4) scikit_learn (machine learning)'. The notebook is in a dark theme. The left sidebar shows a file explorer with a single file. The main area displays a code cell titled 'STEP 1: IMPORT LIBRARIES'. The code imports pandas as pd, matplotlib.pyplot as plt, train_test_split from sklearn.model_selection, LinearRegression from sklearn.linear_model, and mean_squared_error from sklearn.metrics.

```
[ ]  
import pandas as pd  
import matplotlib.pyplot as plt  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression  
from sklearn.metrics import mean_squared_error
```



The screenshot shows a Jupyter Notebook interface with the title '1.4) scikit_learn (machine learning)'. The notebook is in a dark theme. The left sidebar shows a file explorer with a single file. The main area displays a code cell titled 'STEP 2: CREATE DATASET'. The code creates a dictionary 'data' with three keys: 'Student' (a list of 10 student IDs), 'Classes_Attended' (a list of 10 values), and 'Internal_Marks' (a list of 10 values). It then creates a pandas DataFrame 'df' from 'data' and prints it. The output shows a table with 10 rows and 4 columns: 'Student', 'Classes_Attended', and 'Internal_Marks' (the fourth column is empty).

```
[ ]  
data = {  
    'Student': ['S1', 'S2', 'S3', 'S4', 'S5', 'S6', 'S7', 'S8', 'S9', 'S10'],  
    'Classes_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],  
    'Internal_Marks': [35, 30, 42, 46, 50, 55, 60, 65, 70, 75]  
}  
df = pd.DataFrame(data)  
print(df)
```

	Student	Classes_Attended	Internal_Marks	
0	S1	30	35	
1	S2	35	30	
2	S3	40	42	
3	S4	45	46	
4	S5	50	50	
5	S6	55	55	
6	S7	60	60	
7	S8	65	65	
8	S9	70	70	
9	S10	75	75	

1.4) scikit_learn (machine learning)

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

Google Account
THALUPULA SAI TEJA 11249A374
11249a374@kanchiuniv.ac.in

STEP 3: FEATURES & TARGET

```
[ ]
x = df[['Classes_Attended']]
y = df[['Internal_Marks']]
print(x)
print(y)
```

	Classes_Attended
0	30
1	35
2	40
3	45
4	50
5	55
6	60
7	65
8	70
9	75

```
0    35
1    38
2    42
3    46
4    50
5    55
6    60
7    65
8    70
9    75
Name: Internal_Marks, dtype: int64
```

```
[4] ✓ On X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=7
)

[7] ✓ On model = LinearRegression()
model.fit(X_train, y_train)

- LinearRegression
LinearRegression()

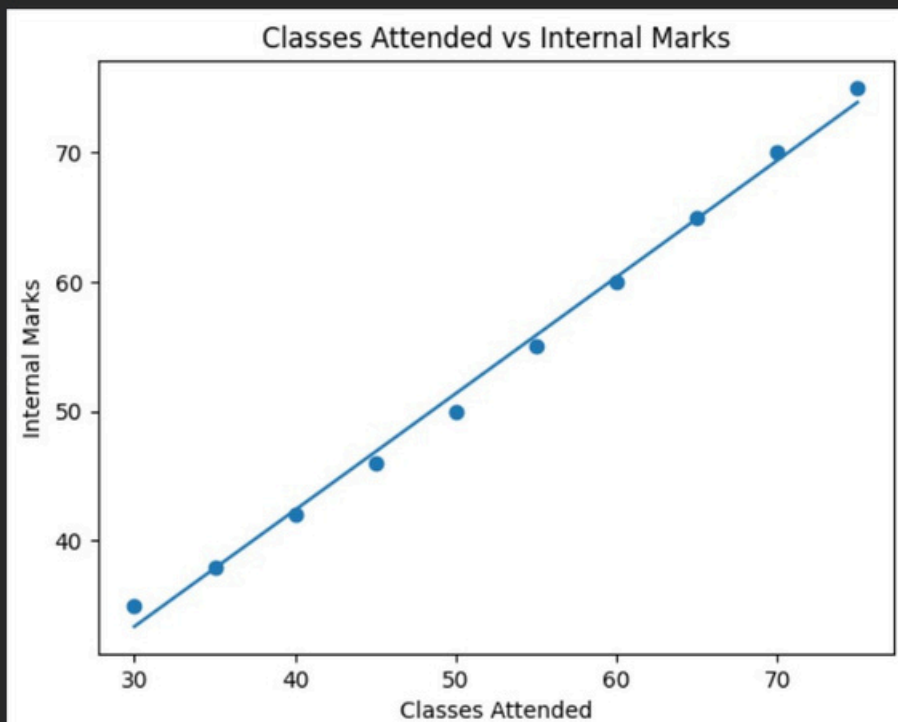
[8] ✓ On predictions = model.predict(X_test)

[9] ✓ On mse = mean_squared_error(y_test, predictions)
print("mse:", mse)
print("Marks per Class:", model.coef_[0])
print("Base Marks:", model.intercept_)

mse: 0.578125
Marks per Class: 0.9
Base Marks: 6.375
```

```
[18]: plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.xlabel("Classes Attended")
plt.ylabel("Internal Marks")
plt.title("Classes Attended vs Internal Marks")
plt.show()
```

output:-



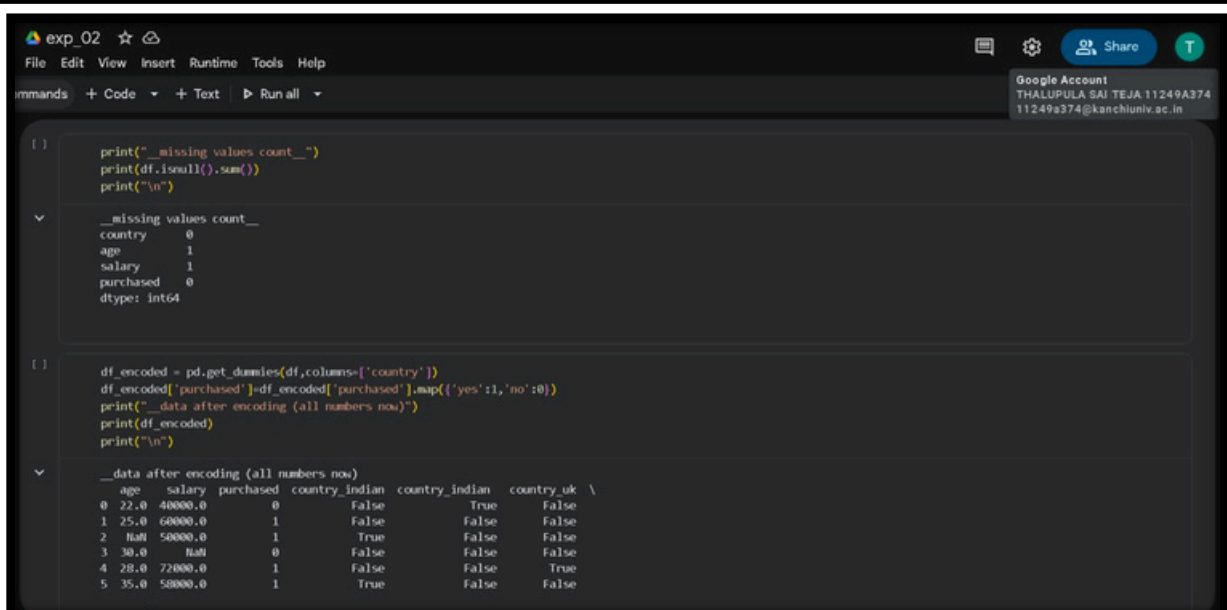
Result: Learned how to apply Scikit-learn for training, testing, and evaluating machine learning models in data science tasks.

EXP.NO:02

DATE:-14/02/2026

Perform Data Exploration and Pre-processing in Python

Aim: To explore datasets and prepare them for analysis by cleaning and transforming the data.

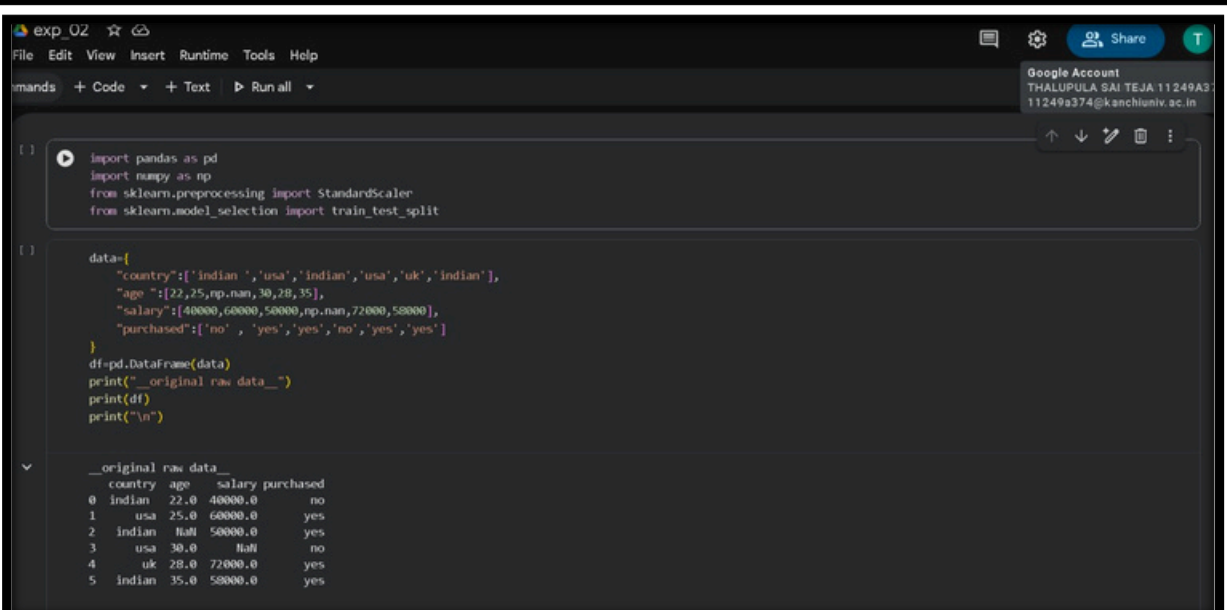


```
[ ]
print("_missing values count_")
print(df.isnull().sum())
print("\n")

__missing values count__
country      0
age          1
salary       1
purchased    0
dtype: int64

[ ]
df_encoded = pd.get_dummies(df,columns=['country'])
df_encoded['purchased']=df_encoded['purchased'].map({'yes':1,'no':0})
print("_data after encoding (all numbers now)")
print(df_encoded)
print("\n")

__data after encoding (all numbers now)
age  salary  purchased  country_indian  country_uk \
0  22.0  40000.0         0             False          True
1  25.0  60000.0         1             False          False
2  30.0  50000.0         1              True          False
3  30.0      NaN         0             False          False
4  28.0  72000.0         1             False          True
5  35.0  58000.0         1              True          False
```



```
[ ]
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

[ ]
data={
    "country":["indian ','usa','indian','usa','uk','indian'],
    "age ":[22,25,np.nan,30,28,35],
    "salary":[40000,60000,50000,np.nan,72000,58000],
    "purchased":["no' ','yes','yes','no','yes','yes']
}
df=pd.DataFrame(data)
print("_original raw data_")
print(df)
print("\n")

__original raw data__
country age  salary purchased
0 indian 22.0 40000.0 no
1 usa 25.0 60000.0 yes
2 indian itali 50000.0 yes
3 usa 30.0 itali no
4 uk 28.0 72000.0 yes
5 indian 35.0 58000.0 yes
```

```
exp_02 ☆
File Edit View Insert Runtime Tools Help
mmands + Code + Text ▶ Run all
Google Account
THALUPULA SAI TEJA 11249A374
11249a374@kanchiuniv.ac.in

country_usa
0 False
1 True
2 False
3 True
4 False
5 False

[]
x= df_encoded.drop('purchased',axis=1)
y=df_encoded['purchased']
scaler=StandardScaler()
X_scaled=scaler.fit_transform(x)
print("__FINAL PROCESSED DATA (ready for ai)__")
print(pd.DataFrame(X_scaled,columns=x.columns).head())

__FINAL PROCESSED DATA (ready for ai)__
age salary country_indian country_indian country_uk \
0 -1.355262 -1.501173 -0.707107 2.236068 -0.447214
1 -0.677631 0.375293 -0.707107 -0.447214 -0.447214
2 NaN -0.562940 1.414214 -0.447214 -0.447214
3 0.451754 NaN -0.707107 -0.447214 -0.447214
4 0.000000 1.501173 -0.707107 -0.447214 2.236068

country_usa
0 -0.707107
1 1.414214
```

```
exp_02 ☆
File Edit View Insert Runtime Tools Help
mmands + Code + Text ▶ Run all
Connect ^

[]
x= df_encoded.drop('purchased',axis=1)
y=df_encoded['purchased']
scaler=StandardScaler()
X_scaled=scaler.fit_transform(x)
print("__FINAL PROCESSED DATA (ready for ai)__")
print(pd.DataFrame(X_scaled,columns=x.columns).head())

__FINAL PROCESSED DATA (ready for ai)__
age salary country_indian country_indian country_uk \
0 -1.355262 -1.501173 -0.707107 2.236068 -0.447214
1 -0.677631 0.375293 -0.707107 -0.447214 -0.447214
2 NaN -0.562940 1.414214 -0.447214 -0.447214
3 0.451754 NaN -0.707107 -0.447214 -0.447214
4 0.000000 1.501173 -0.707107 -0.447214 2.236068

country_usa
0 -0.707107
1 1.414214
2 -0.707107
3 1.414214
4 -0.707107
```

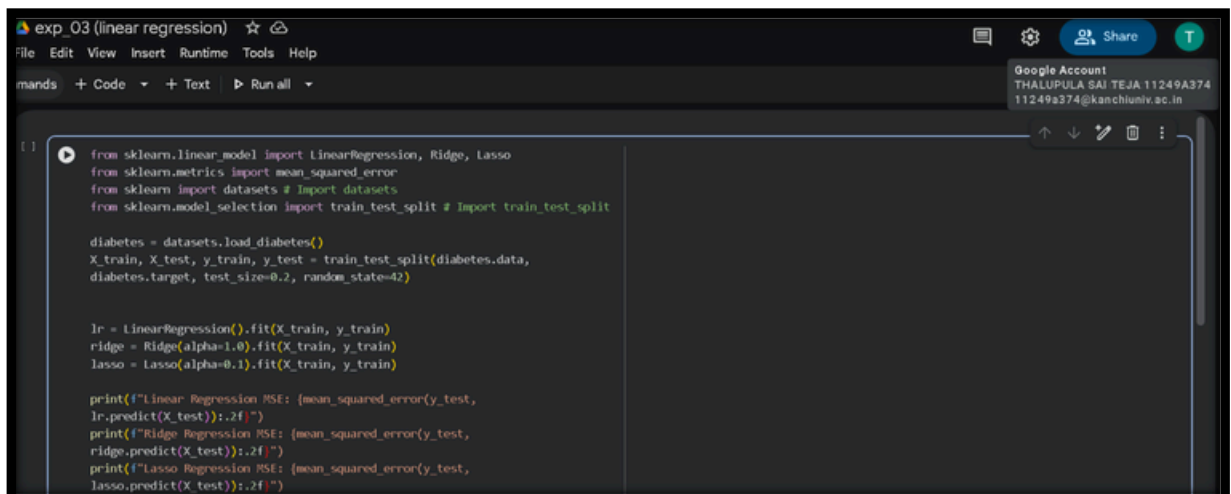
Result: Successfully performed data exploration, handled missing values, encoded categorical features, and scaled data.

EXP.NO:03

DATE:-14/02/2026

Implementation regularized linear regression

AIM:- To implement linear regression with regularization techniques to improve model performance.



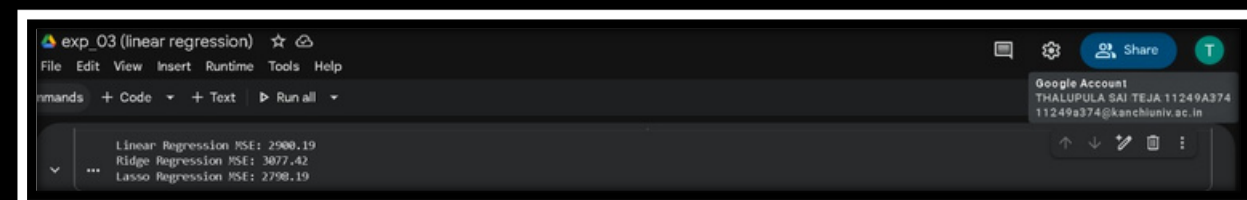
```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error
from sklearn import datasets # Import datasets
from sklearn.model_selection import train_test_split # Import train_test_split

diabetes = datasets.load_diabetes()
X_train, X_test, y_train, y_test = train_test_split(diabetes.data,
diabetes.target, test_size=0.2, random_state=42)

lr = LinearRegression().fit(X_train, y_train)
ridge = Ridge(alpha=1.0).fit(X_train, y_train)
lasso = Lasso(alpha=0.1).fit(X_train, y_train)

print(f"Linear Regression MSE: {mean_squared_error(y_test,
lr.predict(X_test)):.2f}")
print(f"Ridge Regression MSE: {mean_squared_error(y_test,
ridge.predict(X_test)):.2f}")
print(f"Lasso Regression MSE: {mean_squared_error(y_test,
lasso.predict(X_test)):.2f}")
```

Output:-



```
Linear Regression MSE: 2900.19
Ridge Regression MSE: 3077.42
Lasso Regression MSE: 2798.19
```

Result: Built a regularized linear regression model and observed reduced overfitting

EXP.NO:04

DATE:-23/02/2026

Implementation navie bayes classified

AIM:- To implement a Naive Bayes classification model for prediction tasks



```
[ ] from sklearn.metrics import accuracy_score

data = {
    'StudyHours': [5, 2, 8, 1, 6, 3, 7, 4],
    'Attendance': [80, 50, 90, 40, 85, 60, 88, 65],
    'Result': ['Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail']
}

df = pd.DataFrame(data)

X = df[['Studyhours', 'Attendance']]
y = df['Result']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

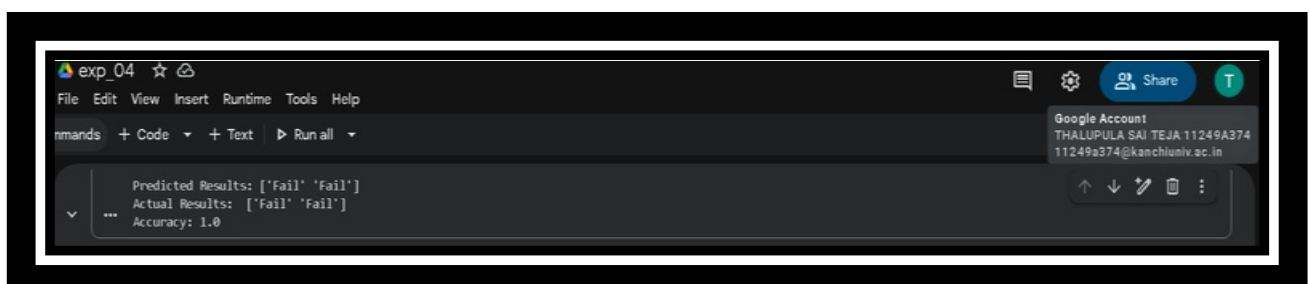
model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Predicted Results:", y_pred)
print("Actual Results: ", y_test.values)
print("Accuracy:", accuracy)
```

output:-



```
Predicted Results: ['Fail' 'Fail']
Actual Results: ['Fail' 'Fail']
Accuracy: 1.0
```

Result: Successfully trained and tested a Naive Bayes classifier and obtained classification results.

EXP.NO:05

DATE:- 2/03/2026

Logistic regression

AIM: To implement logistic regression with regularization for classification.

```
exp 5(logistic regression) ☆ ☁
File Edit View Insert Runtime Tools Help
nmands + Code + Text ▶ Run all

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

normal_model = LogisticRegression(C=1e6, penalty='l2', solver='liblinear')
normal_model.fit(X_train_scaled, y_train)
normal_pred = normal_model.predict(X_test_scaled)
```

```
colab.research.google.com/drive/1QeagQKBU-qJusQSySuCC19vOZic12N9
exp 5(logistic regression) ☆ ☁
File Edit View Insert Runtime Tools Help
Commands + Code + Text ▶ Run all

normal_pred = normal_model.predict(X_test_scaled)
normal_acc = accuracy_score(y_test, normal_pred)

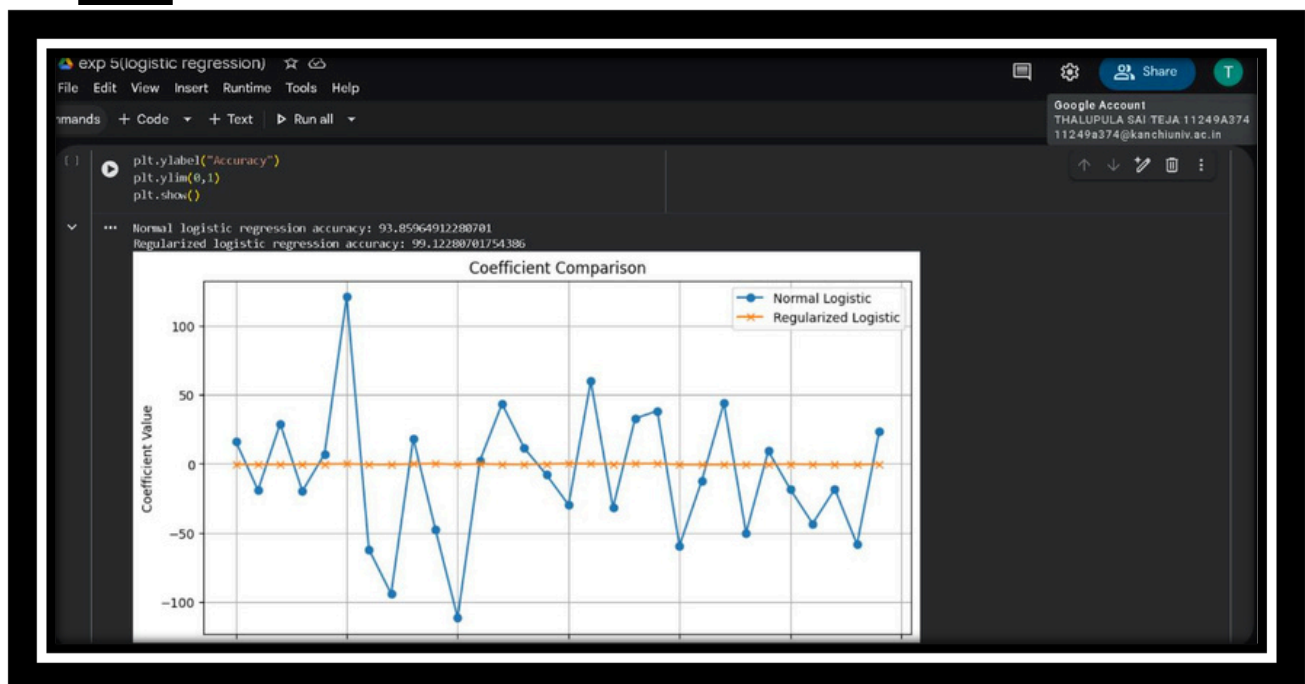
regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='liblinear')
regularized_model.fit(X_train_scaled, y_train)
reg_pred = regularized_model.predict(X_test_scaled)
reg_acc = accuracy_score(y_test, reg_pred)

print("Normal logistic regression accuracy:", normal_acc * 100)
print("Regularized logistic regression accuracy:", reg_acc * 100)

plt.figure(figsize=(10,5))
plt.plot(normal_model.coef_.flatten(), label='Normal logistic', marker='o')
plt.plot(regularized_model.coef_.flatten(), label='Regularized logistic', marker='x')
plt.legend()
plt.title("Coefficient Comparison")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.grid(True)
plt.show()

plt.figure(figsize=(6,4))
models = ['Normal logistic', 'Regularized logistic']
accuracies = [normal_acc, reg_acc]
plt.bar(models, accuracies, colors=['blue', 'orange'])
plt.show()
```

output:-



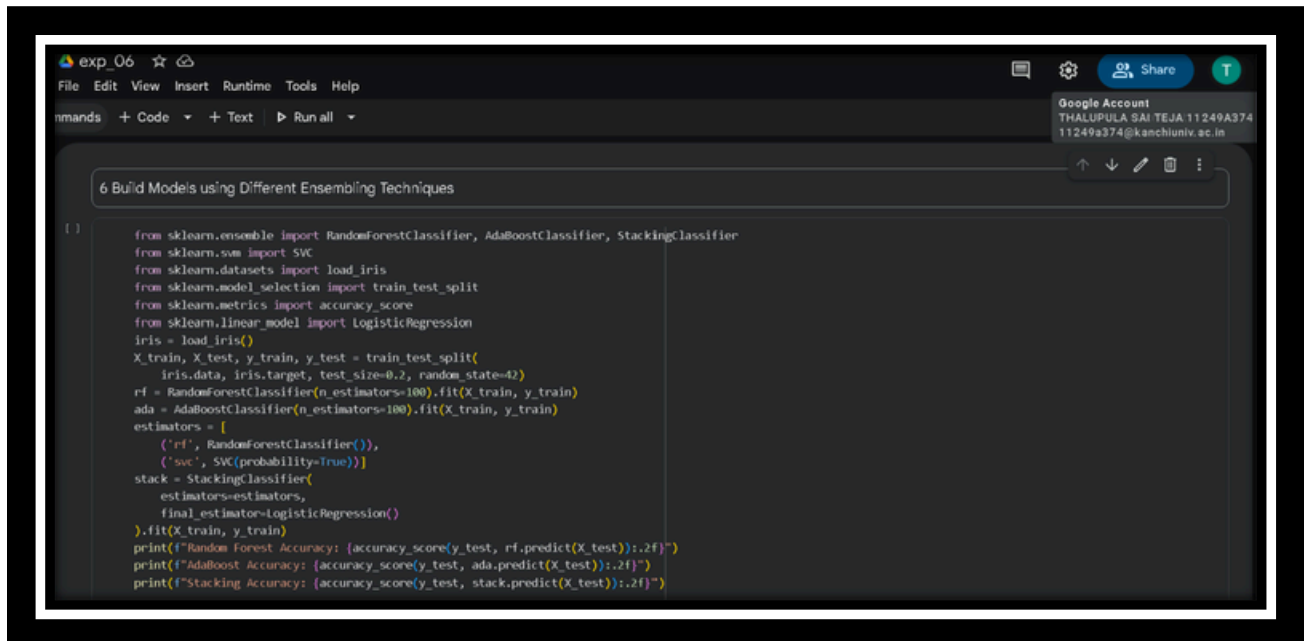
Result: Built a regularized logistic regression model and evaluated its classification accuracy.

EXP.NO:06

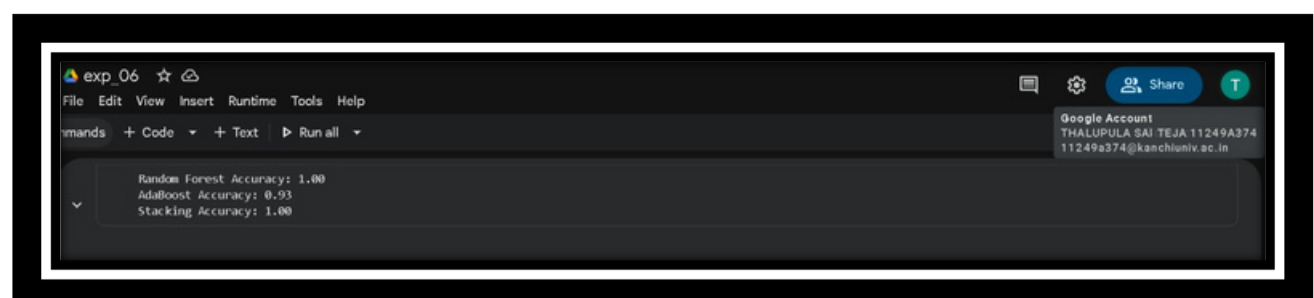
DATE:-09/03/2026

Build models using different ensemble techniques

AIM:- To understand and implement ensemble learning methods.



```
[ ] from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression
iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42)
rf = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)
ada = AdaBoostClassifier(n_estimators=100).fit(X_train, y_train)
estimators = [
    ('rf', RandomForestClassifier()),
    ('svc', SVC(probability=True))]
stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
).fit(X_train, y_train)
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
```



```
Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00
```

Result: Successfully built models using ensemble techniques such as bagging, boosting, or random forest

EXP.NO:07

DATE:-16/03/2026

Build models using decision trees

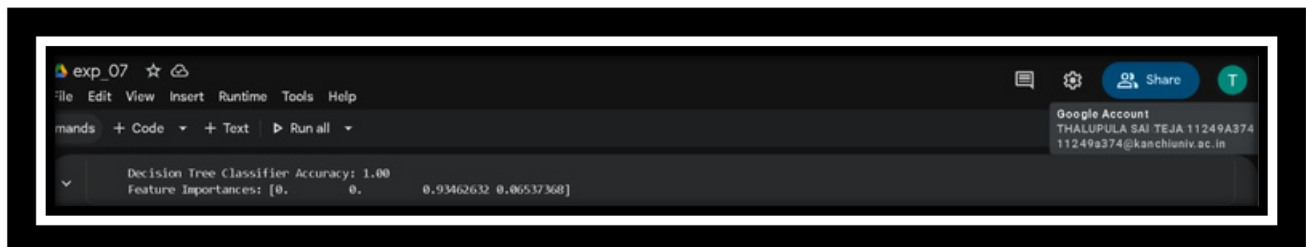
Aim: To construct decision tree models for classification or



```
[ ]
from sklearn.tree import DecisionTreeClassifier
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42)
dt_clf = DecisionTreeClassifier(max_depth=3)
dt_clf.fit(X_train, y_train)
y_pred = dt_clf.predict(X_test)
print(f"Decision Tree Classifier Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print("Feature Importances:", dt_clf.feature_importances_)
```

regression.

Output:-



```
Decision Tree Classifier Accuracy: 1.00
Feature Importances: [0. 0. 0.93462632 0.06537368]
```

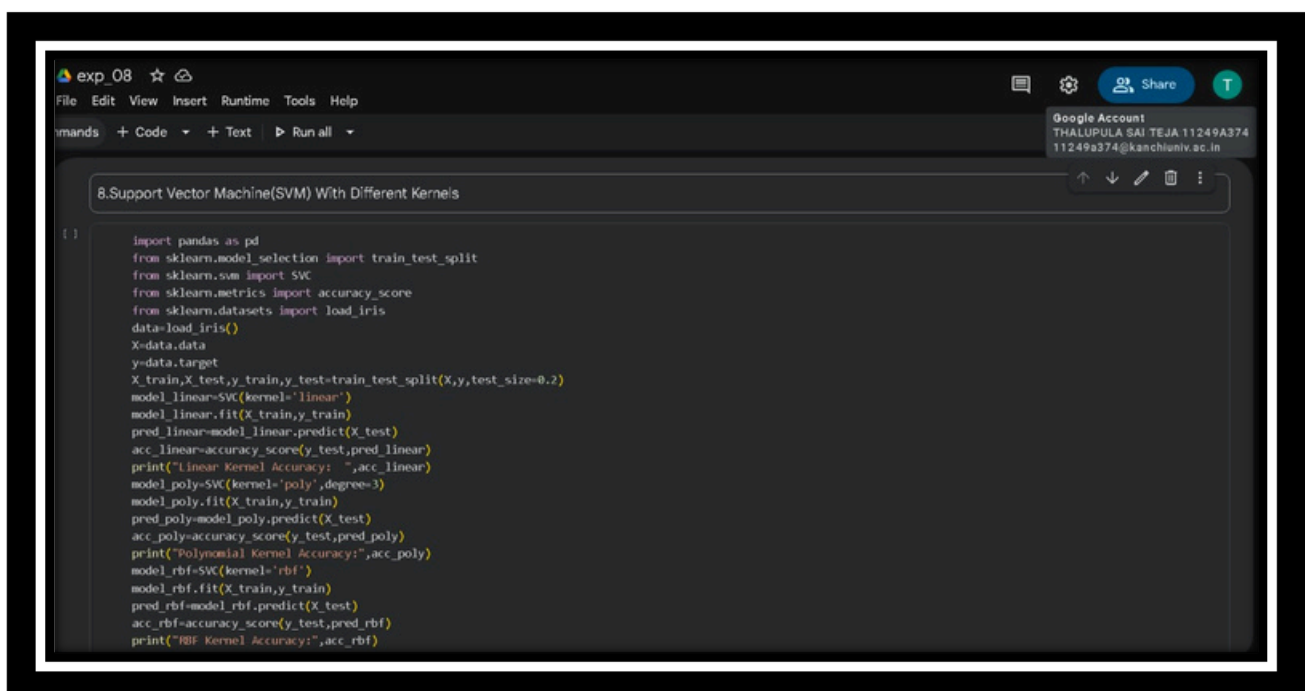
Result: Successfully trained a decision tree model and used it for prediction.

EXP.NO:08

DATE:-23/03/2026

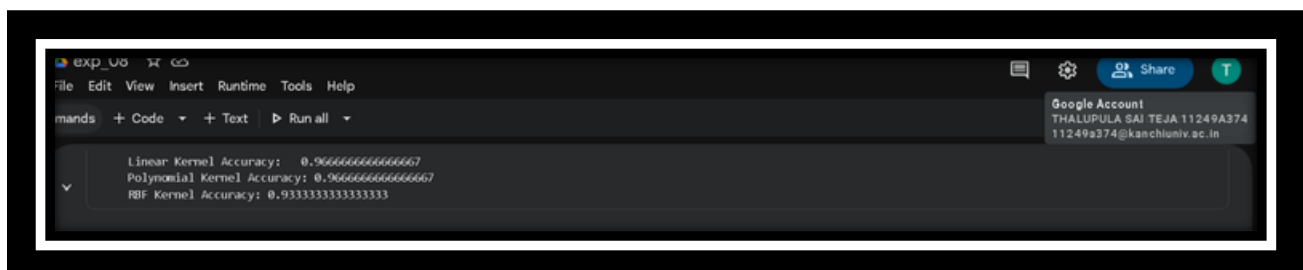
support vector machine (svm) with diffrent kernels

Aim: To study and implement kernel-based machine learning models.



```
[ ]
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
data=load_iris()
X=data.data
y=data.target
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2)
model_linear=SVC(kernel='linear')
model_linear.fit(X_train,y_train)
pred_linear=model_linear.predict(X_test)
acc_linear=accuracy_score(y_test,pred_linear)
print("Linear Kernel Accuracy: ",acc_linear)
model_poly=SVC(kernel='poly',degree=3)
model_poly.fit(X_train,y_train)
pred_poly=model_poly.predict(X_test)
acc_poly=accuracy_score(y_test,pred_poly)
print("Polynomial Kernel Accuracy:",acc_poly)
model_rbf=SVC(kernel='rbf')
model_rbf.fit(X_train,y_train)
pred_rbf=model_rbf.predict(X_test)
acc_rbf=accuracy_score(y_test,pred_rbf)
print("RBF Kernel Accuracy:",acc_rbf)
```

output:-



```
Linear Kernel Accuracy:  0.9666666666666667
Polynomial Kernel Accuracy: 0.9666666666666667
RBF Kernel Accuracy: 0.9333333333333333
```

Result: Built models using different kernel functions and compared their performance.

EXP.NO:09

DATE:- 23/03/2026

Implement K-Nearest Neighbours (KNN) Classification

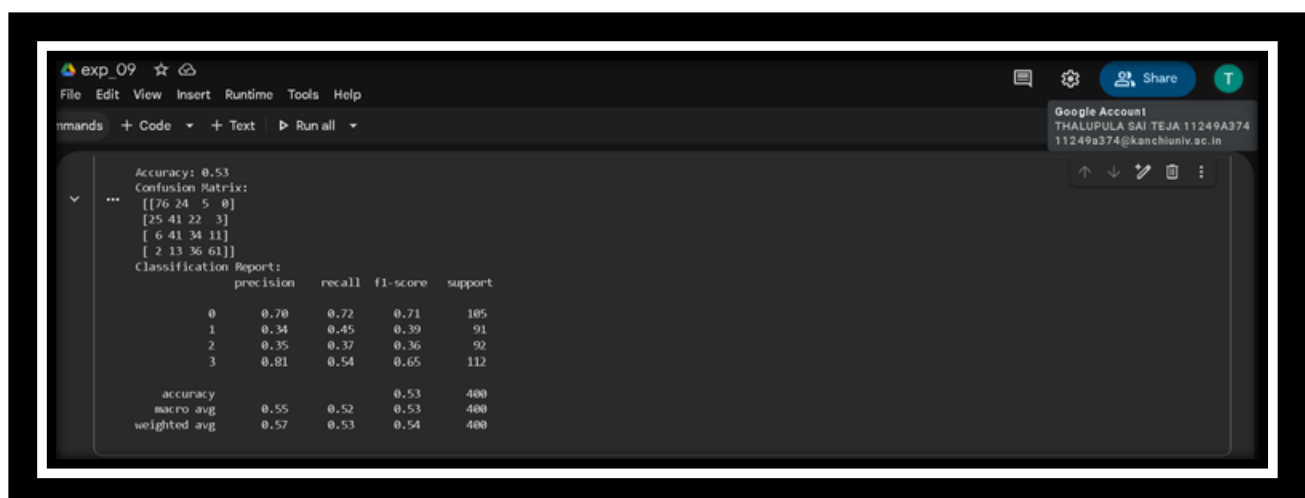
Aim: To implement the KNN algorithm for classification tasks.



```
[ ] import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
data = pd.read_csv("mobile_price.csv")
X = data.drop("price_range", axis=1)
y = data["price_range"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

output:-



```
Accuracy: 0.53
Confusion Matrix:
[[ 76 24  5  0]
 [25 41 22  3]
 [ 6 41 34 11]
 [ 2 13 36 61]]
Classification Report:
              precision    recall  f1-score   support

0               0.70        0.72       0.71         105
1               0.34        0.45       0.39           91
2               0.35        0.37       0.36           92
3               0.81        0.54       0.65         112

 accuracy          0.53         0.53         0.53         400
 macro avg         0.55         0.52         0.53         400
 weighted avg      0.57         0.53         0.54         400
```

Result: Successfully classified data using the KNN algorithm and evaluated the model

EXP.NO:10

DATE:- 23/03/2026

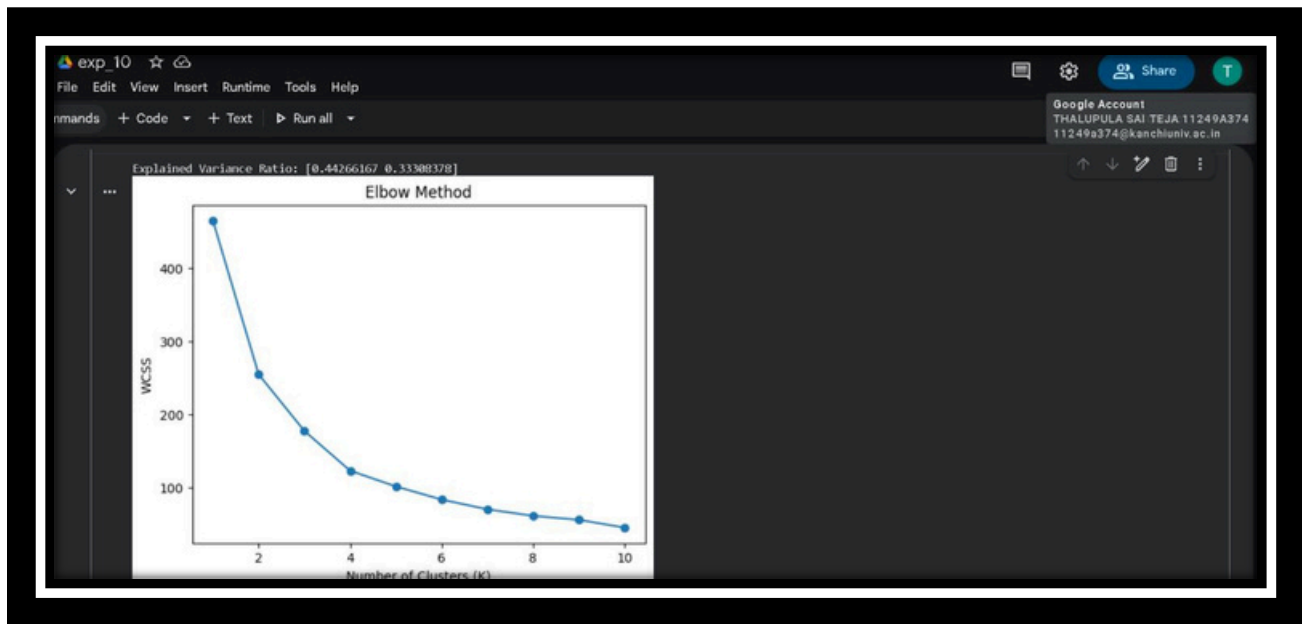
K-Mean Clustering with PCA and Elbow Method

Aim: To perform clustering using K-means and determine the optimal number of clusters using PCA and the elbow method.

```
exp_10
File Edit View Insert Runtime Tools Help
mands + Code + Text Run all
Google Account
THALUPULA SAI TEJA 11249A374
11249a374@kanchiuniv.ac.in

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
data = pd.read_csv("mall customer.csv")
X = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
wcss = []
for i in range(1,11):
    kmeans = KMeans(n_clusters=i, random_state=0)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)
plt.plot(range(1,11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.show()
kmeans = KMeans(n_clusters=5, random_state=0)
clusters = kmeans.fit_predict(X_pca)
plt.scatter(X_pca[:,0], X_pca[:,1], c=clusters)
plt.title("K-Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
```

output:-



Result: Successfully applied K-means clustering, reduced dimensions using PCA, and identified the best cluster count using the elbow method.

